1. Организация

2. Преговор
2.1 Упражнение 1

3. Защо двоична бройна система?

4. Какво е операционна система?
4.1 Упражнение 2

Занятия

- Два блока седмично; във всеки от тях един учебен час теория и един учебен час практика.
- За практическите занятия Ви е необходим *компютър*. В случай, че за някой конкретен това не е така, ще предупреждавам своевременно.

Конспект - сбита версия

0. Въведение в ОС
1. Програмиране на C
2. Памет
3. Процесор
4. Файлови системи
5. Мрежи
6. Сигурност

Бройни системи

Какво е бройна система?

$$14_{(10)} = 1 \cdot 10^1 + 4 \cdot 10^0 \implies 14_{(10)}$$
$$14_{(10)} = 1 \cdot 2^3 + 1 \cdot 2^2 + 1 \cdot 2^1 + 0 \cdot 2^0 \implies 1110_{(2)}$$
$$14_{(10)} = 1 \cdot 3^2 + 1 \cdot 3^1 + 2 \cdot 3^0 \implies 112_{(3)}$$
$$14_{(10)} = 1 \cdot 8^1 + 6 \cdot 8^0 \implies 16_{(8)}$$
$$14_{(10)} = 14 \cdot 16^0 \implies 14_{(16)} \equiv E_{(16)}$$

Бройни системи

$$14_{(10)} = 1 \cdot 2^3 + 1 \cdot 2^2 + 1 \cdot 2^1 + 0 \cdot 2^0 = 1110_{(2)}$$

Бройни системи

$$14_{(10)} = 1 \cdot 2^3 + 1 \cdot 2^2 + 1 \cdot 2^1 + 0 \cdot 2^0 = 1110_{(2)}$$

База

Бройни системи

$$14_{(10)} = 1 \cdot 2^3 + 1 \cdot 2^2 + 1 \cdot 2^1 + 0 \cdot 2^0 = 1110_{(2)}$$

Коефициенти

Бройни системи

$$14_{(10)} = 1 \cdot 2^3 + 1 \cdot 2^2 + 1 \cdot 2^1 + 0 \cdot 2^0 = 1110_{(2)}$$

Позиции

Бройни системи

$$14_{(10)} = 1 \cdot 2^3 + 1 \cdot 2^2 + 1 \cdot 2^1 + 0 \cdot 2^0 = 1110_{(2)}$$

База

Коефициенти

Позиции

От значение е и базата на първоначалната бройна система база.

Бройни системи

- Най-важните бройни системи за нас са двоична и шестнайсетична.
- Първата, заради нейната тясна връзка с хардуера, върху който се изпълняват нашите програми. Ще бележим с префикс `0b`.
- Втората, защото е много лесно преобразуването ѝ към двоична и обратно. *Защо?* Ще бележим с префикс `0x`.

Бройни системи

- Ключовото наблюдение е, че една шестнайсетична цифра описва точно половин байт - т.е. четири бита. Най-голямата шестнайсетична цифра е `f`.

$$\texttt{f}_{(16)} = 1111_{(2)}$$
$$\texttt{ff}_{(16)} = 11111111_{(2)}$$

Бройни системи

- Превръщаме от двоична към шестнайсетична, разделяйки поредицата от битове (започвайки от най-младшия) на групи от по четири (наричаме ги тетрада), и на всяка от получените групи съпоставяме съответната шестнайсетична цифра.
- Обратната посока. Превръщаме от шестнайсетична към двоична, като на всяка цифра съпоставяме група от по четири бита.

Десетични представки

Представките в системата SI умножават мерната единица по степен на 10 - например $1\,\mathrm{km} = 10^3\,\mathrm{m}$, а $1\,\mathrm{ms} = 10^{-3}\,\mathrm{s}$.

Представка	**Множител**
кило (k)	10^3
мега (M)	10^6
гига (G)	10^9
тера (T)	10^{12}
пета (P)	10^{15}

Представка	**Множител**
мили (m)	10^{-3}
микро (μ)	10^{-6}
нано (n)	10^{-9}
пико (p)	10^{-12}
фемто (f)	10^{-15}

Няколко важни константи

Уговорка. За нас един килобайт е 1024 байта. Аналогично имаме

$$1\,\mathrm{KB} = 1024\,\mathrm{B}$$
$$1\,\mathrm{MB} = 1024\,\mathrm{KB}$$
$$1\,\mathrm{GB} = 1024\,\mathrm{MB}$$
$$1\,\mathrm{TB} = 1024\,\mathrm{GB}$$
$$1\,\mathrm{PB} = 1024\,\mathrm{TB}$$

Няколко важни константи

Няма да се придържаме към системата SI, която определя един килобайт като 1000 байта, а 1024 байта нарича кибибайт. Важните константи са:

$$1\,\mathrm{KB} = 2^{10}\,\mathrm{B}$$
$$1\,\mathrm{MB} = 2^{20}\,\mathrm{B}$$
$$1\,\mathrm{GB} = 2^{30}\,\mathrm{B}$$
$$1\,\mathrm{TB} = 2^{40}\,\mathrm{B}$$
$$1\,\mathrm{PB} = 2^{50}\,\mathrm{B}$$

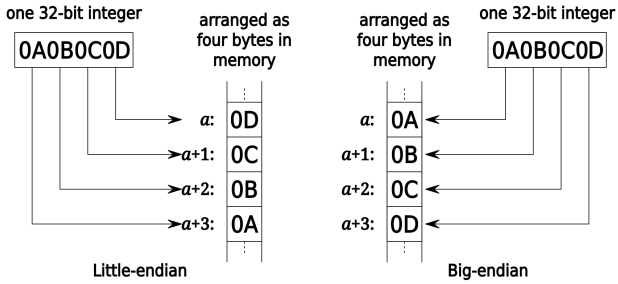
Няколко важни константи

Тяхното помнене е доста удобно, когато трябва бързо да превърнете произволно число - било то отместване във файл или големина на описвано множество от побитова карта, в размер.

Индианци и яйца (Endianness)

- *Пример.* Нека имаме целочислено число, съставено от четири байта (`uint32_t`). В паметта то се представя чрез поредица от съседни байтове - това е неговият *запис*. Програмистът влага на тези байтове *смисъл* на число, представено в двоична бройна система.
- За реда на байтовете (записа) има два общоприети варианта - отдясно-наляво или отляво-надясно; т.е. младшите битове се намират съответно или най-вдясно, или най-вляво.

Индианци и яйца (Endianness)



Фигура: Илюстрация на endianness при 4-байтово число.

Индианци и яйца (Endianness)

- Когато байтовете на данните са разположени отдясно-наляво, казваме, че се използва big endian. Обратно, ако са отляво-надясно - little endian.[1]
- При повечето архитектури (x86, ARM) се използва little-endian.
- При комуникация през мрежата се използва big-endian (още наричан network order).

[1] Терминът *endianness* произлиза от творбата „Пътешествията на Гъливер“ на Джонатан Суифт, в един епизод от която две групи лилипути имат спор за това от кой край трябва да се чупи сварено яйце - дали от малкия (little end), или от големия (big end). Оттам идват и съответно little-endian и big-endian. Действителна връзка с индианци няма. ☺

Упражнение

- Превърнете 0b111011110011 в шестнайсетична бройна система. Отг. 0xef3
- Ако горното число беше размер на файл, то колко щеше да бъде голям той? Отг. 3KB 755B
- Ако горното число беше записът в little-endian, то колко щеше да бъде размерът на файла? Отг. 0b001111111110 = 0x3fe = 1022B
- 0x8000000, 0x1000, 0x400, 0x10000. Отг. 128MB, 4KB, 1KB, 64KB
- Използвайте програма, написана на C, за да проверите дали машината Ви използва little- или big-endian за 4-байтово целочислено число. *Подсказка.* В gdb използвайте команда `x/4b &a`, където a е вашата променлива.

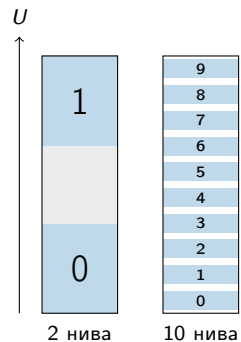
Защо двоична бройна система?

- Не винаги е било така. Машините на Бабидж (XIX в.), Harvard Mark I (1944) и ENIAC (1945) смятат в *десетична* система - за всяка цифра има колело или брояч с 10 положения.
- По същото време Z3 на Конрад Цузе (1941) и ABC на Джон Атанасов и Клифърд Бери (1942) смятат в *двоична*.[2]
- След доклада на фон Нойман за EDVAC (1945) почти всички компютри са двоични. *Защо?*

[2] Бащата на Джон Атанасов, Иван Атанасов, е българин от село Бояджик.

Защо точно две състояния?

- Сигналът е напрежение - непрекъсната величина, върху която винаги има шум.
- При 2 нива всяка стойност има широк интервал и шумът трудно ги обърква. При 10 нива интервалите са много по-тесни.
- Транзисторът не е „двоичен“ - в цифровите схеми го използваме като ключ, напълно отворен или затворен.



Двоичната логика е проста

- Две състояния са истина и лъжа - аритметиката се свежда до булева алгебра (Шанън, 1937), а И, ИЛИ и НЕ се изграждат от няколко транзистора.
- Всичко, което обработваме - числа, текст, изображения, звук, програми - може да се запише като поредица от битове.
- Не е догма. Сетун (МГУ, 1958) е троичен компютър, флаш паметите MLC/TLC/QLC пазят по 2-4 бита в клетка (4-16 нива на заряд), а PCIe 6.0 предава по 2 бита наведнъж (PAM4, 4 нива). Там допълнителните грешки се поемат от кодове за корекция.

Цели числа без знак

- С N бита записваме 2^N различни стойности. Без знак (unsigned) това са числата от 0 до $2^N - 1$.

Бита	Обхват
8	от 0 до 255
16	от 0 до 65 535
32	от 0 до 4 294 967 295
64	от 0 до $\approx 1.8 \cdot 10^{19}$

- Броят битове е фиксиран. Пренос, който не се побира, се губи - смятаме по модул 2^N. В 8 бита $255 + 1 = 0$, като километраж на кола.

Цели числа в C

Тип	Минимум по стандарт	Linux x86-64	Windows x64
`char`	8 бита	8	8
`short`	16 бита	16	16
`int`	16 бита	32	32
`long`	32 бита	64	32
`long long`	64 бита	64	64

- Стандартът гарантира само минимални размери, реалните зависят от платформата - проверете със `sizeof`.
- Когато размерът е важен, използвайте типовете от `<stdint.h>`: `int8_t`, `uint16_t`, `int32_t`, `uint64_t` и т.н.

Отрицателни числа - първи опит

- Най-простата идея: най-старшият бит е знак, а останалите - големина (на англ. sign and magnitude).

$$+5 = 0000\,0101 \qquad -5 = 1000\,0101$$

- Два проблема:
  - две нули - $0000\,0000$ $(+0)$ и $1000\,0000$ (-0);
  - обикновеното събиране вече не работи:
    $0000\,0101 + 1000\,0101 = 1000\,1010 = -10$, а не 0.

Допълнителен код

- В допълнителен код (на англ. two's complement) най-старшият бит получава *отрицателно* тегло -2^{N-1}:

$$1111\,1011 = -128 + 64 + 32 + 16 + 8 + 2 + 1 = -5$$

- Смяна на знака: обръщаме всички битове и добавяме 1.

$$5 = 0000\,0101 \longrightarrow 1111\,1010 \longrightarrow 1111\,1011 = -5$$

- Събирането работи без промяна - преносът навън се изхвърля:

$$0000\,0101 + 1111\,1011 = (1)\,0000\,0000 = 0$$

Допълнителен код

- Има само една нула, а една и съща схема събира числа със и без знак.
- Обхватът е от -2^{N-1} до $2^{N-1}-1$ - за 8 бита от -128 до 127.
- Старшият бит все още показва знака (1 значи отрицателно), но не е „просто знак“ - -1 е 1111 1111, а не 1000 0001.
- Затова `(unsigned char)-1 == 255`: битовете са същите, само ги четем по различен начин.

Цели числа в C - особености

- От C23 стандартът изисква допълнителен код. Преди е допускал и други представяния, но на практика всички платформи използват него.
- Аритметиката без знак е по модул 2^N, а препълване при числа със знак е *недефинирано поведение* - компилаторът приема, че не се случва.
- Дали `char` е със знак, зависи от платформата: на x86 обикновено е, на ARM Linux - не. Когато е важно, пишете `signed char` или `unsigned char`.
- Двоичните литерали (`0b1010`) са в стандарта от C23, а дотогава са разширение на GCC и Clang.

Дроби - позиции след запетаята

- Позиционният запис продължава и надясно от запетаята - с отрицателни степени на основата:

$$0.625_{(10)} = 6 \cdot 10^{-1} + 2 \cdot 10^{-2} + 5 \cdot 10^{-3}$$
$$0.101_{(2)} = 1 \cdot 2^{-1} + 0 \cdot 2^{-2} + 1 \cdot 2^{-3} = 0.5 + 0.125 = 0.625$$

- Всяка позиция надясно е два пъти по-малка от предишната:

$$2^{-1} = 0.5 \qquad 2^{-2} = 0.25 \qquad 2^{-3} = 0.125 \qquad 2^{-4} = 0.0625$$

- Пример: $5.75 = 4 + 1 + 0.5 + 0.25 = 101.11_{(2)}$

Дроби - от десетична към двоична

- Умножаваме дробната част по 2. Цялата част на резултата е следващият бит, а с дробната продължаваме.
- *Защо?* Умножението по 2 мести двоичната запетая с една позиция надясно - първият бит след нея „излиза“ пред нея.

Пресмятане	**Бит**
$0.625 \times 2 = 1.25$	1
$0.25 \times 2 = 0.5$	0
$0.5 \times 2 = 1.0$	1

$$\Longrightarrow \; 0.625 = 0.101_{(2)}$$

- Спираме, когато дробната част стане 0.

Безкрайни двоични дроби

Пресмятане	Бит
$0.1 \times 2 = 0.2$	0
$0.2 \times 2 = 0.4$	0
$0.4 \times 2 = 0.8$	0
$0.8 \times 2 = 1.6$	1
$0.6 \times 2 = 1.2$	1
$0.2 \times 2 = 0.4$	0
...	

$$0.1 = 0.0\overline{0011}_{(2)}$$

- Повтаря се безкрайно - също като $\frac{1}{3} = 0.333\ldots$
- Краен двоичен запис има само дроб със знаменател степен на 2: $\frac{1}{2}, \frac{3}{4}, \frac{5}{8}$ - да; $\frac{1}{10}, \frac{1}{3}$ - не.
- Компютърът не може да запише 0.1 точно - пази приближение.

Къде да сложим запетаята?

- *Фиксирана запетая* - например 16 бита за цялата част и 16 за дробната. Просто е, но обхватът е малък: от ≈ 0.000015 до $\approx 65\,536$.
- Във физиката имат същия проблем и го решават с научен запис: $6.022 \cdot 10^{23}$, $1.6 \cdot 10^{-19}$. Значещите цифри и порядъкът се пазят поотделно.
- *Плаваща запетая* - научен запис, но с основа 2:

$$5.75 = 101.11_{(2)} = 1.0111_{(2)} \cdot 2^2$$

- Нормализираното число винаги започва с 1, затова тази единица не се записва (неявен бит).

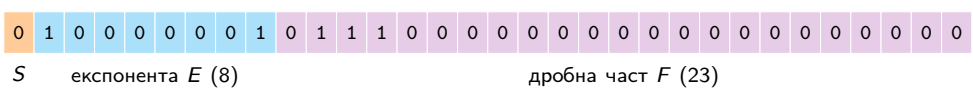
IEEE 754 - float (32 бита)

$$\text{стойност} = (-1)^S \cdot 1.F_{(2)} \cdot 2^{E-127}$$

- E се пази без знак. Отместването 127 позволява отрицателни степени: $E = 127$ означава 2^0, $E = 1$ означава 2^{-126}.
- $E = 0$ и $E = 255$ са запазени - за нулата, много малки (денормализирани) числа, $\pm\infty$ и NaN.
- `double` (64 бита): 1 + 11 + 52 бита, отместване 1023.

Пример - 5.75 като float

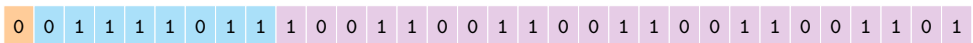
1. Двоично: $5.75 = 101.11_{(2)}$
2. Нормализираме: $1.0111_{(2)} \cdot 2^2$
3. Знак: $S = 0$, защото числото е положително.
4. Експонента: $E = 2 + 127 = 129 = 1000\,0001_{(2)}$
5. Дробна част: битовете след запетаята, 0111, допълнени с нули до 23 бита.



$$= 0\text{x}40\text{B}80000$$

А 0.1?

- $0.1 = 1.1001\,1001\,1001\ldots_{(2)} \cdot 2^{-4}$ - дробната част е безкрайна, затова след 23-тия бит закръгляме.



$$= \texttt{0x3DCCCCCD}$$

- Записаното число е 0.100000001490116119384765625 - близо до 0.1, но не 0.1.

Дроби - какво следва от това

- В `double` $0.1 + 0.2 == 0.3$ е лъжа - сумата е 0.30000000000000004. Пробвайте в любимия си език!
- `float` пази около 7 значещи десетични цифри, `double` - около 15-16.
- Не сравнявайте дроби с ==. Проверявайте дали разликата е достатъчно малка: `fabs(a - b) < 1e-9`.
- Не пазете пари във `float` - пазете цели стотинки в целочислен тип.
- Специални стойности: $1.0/0.0 = +\infty$, $0.0/0.0 = \text{NaN}$, и NaN не е равно дори на себе си.

Символи - ASCII

- Компютърът не знае какво е буква. Всеки символ е число, а таблица определя кое число коя буква е.
- ASCII (1963): 7 бита, 128 символа - латински букви, цифри, препинателни знаци и управляващи символи (например `'\n'` е 10).
- В C `char` е просто малко цяло число: `'A' == 65 == 0b1000001`, `'a' == 97 == 0b1100001`.
- Таблицата е подредена нарочно: малката и главната буква се различават в един бит (`'a' - 'A' == 32`), а стойността на цифра получаваме с `c - '0'`.

Символи - кодови таблици

- ASCII използва 7 от 8-те бита. Останалите 128 стойности всеки използва за своите букви - само за кирилица има CP1251, KOI8-R, ISO 8859-5 и други.
- Един и същ байт е различна буква в различните таблици:

Байт	**Таблица**	**Символ**
`0xC0`	CP1251	А
`0xC0`	KOI8-R	ю
`0xC0`	ISO 8859-5	Р

- Текст, прочетен с грешната таблица, става безсмислица („кракозябри“, на англ. mojibake).

Символи - Unicode и UTF-8

- Unicode дава на всеки символ от всички писмености уникален номер (code point) от U+0000 до U+10FFFF - над 1,1 милиона възможни, от които над 150 000 вече са заети.
- В началото (1991) Unicode е 16-битов, но 65 536 символа се оказват малко и през 1996 г. е разширен.
- UTF-8 записва всеки номер с 1 до 4 байта, а ASCII символите остават същите единични байтове:

A	U+0041	`41`
А (кирилско)	U+0410	`D0 90`
емоджи „усмихнато лице“	U+1F600	`F0 9F 98 80`

- В C `strlen("Ж") == 2` - `strlen` брои байтове, не символи.

Видове софтуер

Какво е системно програмиране?

Видове софтуер

Какво е системно програмиране?

Компонент	**Интерфейс**	**Функции**
Бизнес модул	Към потребителя - конзола, графично	„Бизнес“
Библиотеки за общо предназначение	Прост	Общи - низове, памет, математика и т.н.
Библиотеки за достъп до хардуер	Логическо устройство	Управление на конкретен хардуер

Какво е операционна система?

- Операционната система (ОС) е софтуер, който споделя и разпределя ресурсите на хардуера между различни програми.
- Тези програми получават възможност от ОС да не се съобразяват с останалите агенти на системата, а само със самата ОС.
- Споделянето на ресурсите се осъществява чрез *изолация* и/или чрез *заявки*.
- *Изолацията* е метод, чрез който ОС показва даден ресурс само и единствено на неговия *собственик*.

Какво е операционна система?

- *Методът на заявките* изисква изрично обръщения на всички програми към един и същ ресурс едновременно. Този вид споделяне бива два вида - *последователно* (или блокиращо, или синхронно) и *паралелно* (или асинхронно). При първото програмата блокира, докато не завърши изпратената заявка, докато при второто заявката се поставя в опашка за изпълнение и програмата няма контрол в какъв момент точно ще бъде изпълнена.

Какво е операционна система?

Ресурс	**Абстракция**	**Метод на споделяне**
Процесор	Нишки	Изолация (и времеделене)
Памет	Адресно пространство	Изолация
Диск	Файлова система	Заявки
Мрежа	Сокети	Заявки
Принтер	Логически принтер	Заявки (асинхронни)

Какво е операционна система?

Изискванията към една ОС още включват -

- Скорост - цената за изпълнение на ОС трябва да бъде минимална.
- Надеждност - от издръжливостта на ОС зависят и приложенията.
- Изолация между изпълняващите се приложения с цел сигурност. Същевременно, начин за споделяне на данни между приложения с цел кооперация.
- Абстрахиране на конкретния хардуер с цел преносимост.
- Удобен интерфейс, за да бъде лесна за употреба.

Структура на ОС

След като сме описали задачите, които една ОС трябва да решава, нека опитаме да си представим как изглежда горе-долу една такава.

Нека разгледаме един крайно опростен пример. Имаме приложения, които искат да пишат по диска, за да запазват потребителски данни. ОС, в която се изпълняват, предоставя интерфейс за работа с диска - приложенията извикват `disk_write(sector, data)` и ОС се грижи за магията.

Имаме две непосредствени следствия-проблеми. От една страна това означава, че приложение A може да реши да

Структура на ОС

използва сектор 42 и приложение Б да си хареса същия сектор. Така те ще бъдат в постоянно състояние на *състезание* (на англ. race condition) за данните на сектор 42.

От втора страна, щом това е просто функция, която може да се извика от всеки, то системата е много лесна за компрометиране - произволна програма може да изтрие данните на потребителя. Със сигурност нежелан ефект!

Достигаме до много важен извод - ОС **не** е просто библиотека с функции, които се „грижат за хардуера“. *Откъде точно произлиза разликата обаче?*

Структура на ОС

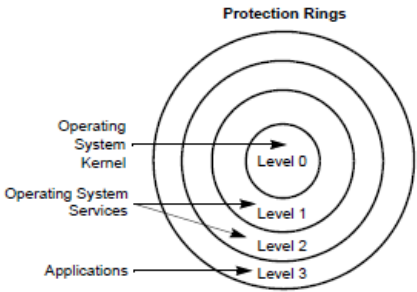
Разликата е, че не всяка програма трябва да може да пише по диска каквото и където си пожелае. Това трябва да бъде допустимо при силни ограничения - все пак не бива да го забраняваме; дискът наистина е за писане. *Как решаваме този проблем?*

Понеже разбираме, че трябва дискът да бъде споделен между множество програми, си спомняме, че това се случва чрез *заявки* към *абстракцията* на диска, а именно - *файловата система*. Така решаваме единия проблем.

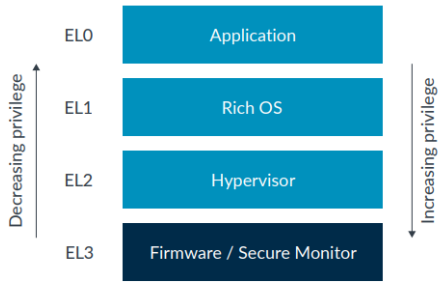
Структура на ОС

Вторият проблем, който представихме, се отнася до сигурността на ОС. Решаваме го по следния начин. Въвеждаме понятието ниво на привилегия (на англ. privilege level или още protection rings). Това е свойство на изпълняваща се програма - казваме, че се изпълнява с някакво ниво на привилегия.

Структура на ОС



Фигура: x86 Нива на привилегия



Фигура: ARM Нива за изключения

Структура на ОС

И тогава решението на проблема ни е много просто - за да изпълнява `disk_write()` (от предходния ни пример), изискваме програмата да се изпълнява с ниво на привилегия, което позволява това.

Както уточнихме вече обаче приложенията не се изпълняват с ниво на привилегия, което позволява това. И това е по дизайн.

Нуждаем се от метод за „промяна на нивото на привилегия“ - изпълняваме си приложението, вършейки позволени операции (математически операции, работа с низове, взаимодействие с потребител), и в някакъв момент искаме да извикаме

Структура на ОС

„забранената“ функция `disk_write()`. След като функцията приключи работата си, искаме нивото на привилегия да се смени отново - този път към началното си състояние.

Този механизъм се нарича системно извикване (на англ. system call или накратко syscall). Чрез него потребителско приложение инициира работа за ОС. Казваме, че приложението се изпълнява в потребителски режим (на англ. user mode), а ОС - в режим на ядрото (на англ. kernel mode). Онази част от ОС, която се изпълнява в режим на ядрото, наричаме ядро (на англ. kernel). При изпълнение в режим на ядрото програмата има

Структура на ОС

достъп до цялата системна памет и всички инструкции на процесора, докато при потребителския режим ОС се налагат ограничения.

Нека имаме едно примерно системно извикване `S()`. Технологията на извикване на `S()` е следната. Когато потребителско приложение A желае да изпълни `S()`, то трябва да задейства софтуерно прекъсване „капан“ (на англ. trap software interrupt), при което процесорът ще съхрани състоянието на A, ще премине към режим на ядрото и ще премине към изпълнение на обработка на софтуерното прекъсване. При

Структура на ОС

обработката на софтуерното прекъсване ще се изпълни работата на `S()`, след което процесорът ще изпълни специална инструкция за излизане от капана, ще възвърне състоянието на приложението и то ще продължи работата си.

Забелязваме обаче един важен детайл. При извикването на `S()` не можем просто да посочим неговия адрес. *Защо?*

Например, ако желаем да отворим файл, за който нямаме права, можем да посочим адрес, съответстващ на функцията след проверката за правоспособност. Така получаваме способност за отваряне на произволен файл, заобикаляйки

Структура на ОС

всякакви проверки за сигурност. Това се нарича уязвимост (на англ. vulnerability).

Решението на този проблем е просто. Ядрото използва фиксирана таблица за своите системни извиквания. Потребителско приложение уточнява единствено индекса на системното извикване, което иска да се изпълни, а ядрото намира адреса на съответстващата функция при обработката на софтуерното прекъсване.

Така изяснихме общата постановка на комуникация между ОС и приложенията, които ОС управлява.

Структура на ОС

Примери за системни извиквания в Linux са `write`, `mmap`, `shmat` и `dup`. Общо са около 500. Може да се запознаете по-обстойно с всеки от тях, използвайки командата `man 2 <syscall>`. Можете да разгледате системните извиквания, които Linux ядрото предоставя, търсейки макрото `SYSCALL_DEFINE`*x*, където *x* е цяло число от 0 до 6 (броя параметри) в хранилището на кода - torvalds/linux.git или в индексиращата система Bootlin. Да отбележим, че когато извикаме например `write` в своя програма това **не** е системното извикване, както очакваме. Всъщност това е функция, дефинирана в стандартната

Структура на ОС

библиотека `glibc`, която извършва описания от нас процес с поставянето на число на определено място и предизвикване на софтуерно прекъсване. Такива функции се наричат обвивка (на англ. wrapper или jacket).

В MacOS ситуацията е подобна на тази в Linux, поради споделения им произход. Можете да разгледате системните извиквания в хранилището на кода - apple/xnu.git. Там вместо подход с макроси, използват генератор на код.

В Windows обичайно се използват различни имена на понятията, но фундаментално идеята е същата. Наместо

Структура на ОС

системно извикване, популярният термин е системна услуга (на англ. system service). Технологията на извикване е сходна - числото, подадено от извикващия се използва от *диспечера на системни услуги*, за да индексира таблица със самите функции, имплементиращи системното извикване. Самите системни извиквания са недокументирани официално, но до известна степен следват т.нар. Native API, имплементиран в `NTDLL.dll`, което е „най-ниската точка“, изпълняваща се в потребителски режим. Примери за системни услуги в Windows са `NtReadFile` и `NtWaitForSingleObject`.

Упражнение

Забележка: Понеже все още не сме преминали през темите относно C, задачките ще изискват само най-елементарни познания и вярвам, че ще се справите ☺.

- Напишете програма, която приема от стандартния си вход три аргумента - име на файл, отместване във файла и низ. Използвайки обвивките на системните извиквания за работа с файл - ОТВАРЯНЕ, ПИСАНЕ и ЗАТВАРЯНЕ, програмата записва на посоченото отместване данните във файла и приключва.

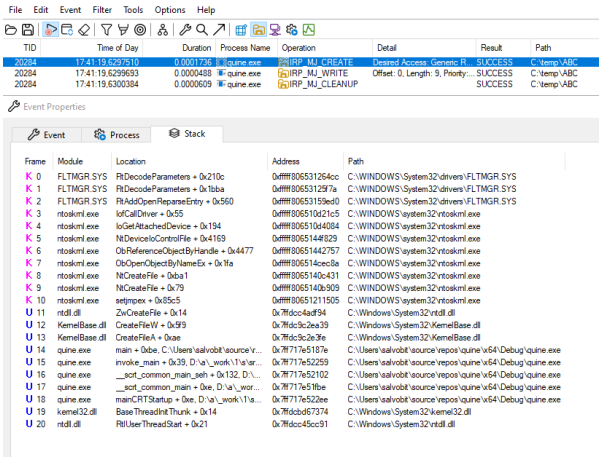
Отг. Под Linux и Mac OS използвайте open, write и close, а под Windows - CreateFileA, WriteFile и CloseHandle.

Упражнение

- Използвайте програма за проследяване на системните извиквания, за да разгледате хода на програмата си. В Windows използвайте `Procmon`, в Mac OS - `sudo dtruss`, а в Linux `strace`. Ако имате други предпочитания, нямам против.
- *Забележка (Windows):* Това, което се вижда в колоната „Operation“ в `Procmon` не са системните услуги. Същевременно при *<десен клик>* $\rightarrow$ *Stacks* се виждат ясно всички извиквания, които се пораждат от функцията, както и това в какъв момент точно се преминава към режим на ядрото.

```
1377/0x3ebc:      3829       2      1 mprotect(0x104AEC000, 0xC8, 0x1)     = 0 0
1377/0x3ebc:      3831       2      0 mprotect(0x104AFD000, 0x1000, 0x3)    = 0 0
1377/0x3ebc:      3834       2      0 mprotect(0x104AFD000, 0x1000, 0x1)    = 0 0
1377/0x3ebc:      4007       2      0 issetugid(0x0, 0x0, 0x0)     = 0 0
1377/0x3ebc:      4336       6      3 getentropy(0x7FF7BB409510, 0x20, 0x0)      = 0 0
1377/0x3ebc:      4463      15     12 getattrlist("/Users/salvobit/Desktop/a.out\0", 0x7FF7BB409CB0, 0x7FF7BB409CCC)     = 0 0
1377/0x3ebc:      4483      18     17 access("/Users/salvobit/Desktop\0", 0x4, 0x0)      = 0 0
1377/0x3ebc:      4497      14     12 open("/Users/salvobit/Desktop\0", 0x0, 0x0)      = 3 0
1377/0x3ebc:      4500       3      1 fstat64(0x3, 0x7F90A7704490, 0x0)      = 0 0
1377/0x3ebc:      4502       2      0 csrctl(0x0, 0x7FF7BB409ECC, 0x4)     = 0 0
1377/0x3ebc:      4504       3      1 fcntl(0x3, 0x32, 0x7FF7BB409BC0)     = 0 0
1377/0x3ebc:      4507       2      1 close(0x3)     = 0 0
1377/0x3ebc:      4521      10      8 open("/Users/salvobit/Desktop/Info.plist\0", 0x0, 0x0)     = -1 Err#2
1377/0x3ebc:      4539       3      1 proc_info(0x2, 0x561, 0xD)     = 64 0
1377/0x3ebc:      4555       3      1 csops_audittoken(0x561, 0x10, 0x7FF7BB409F10)      = -1 Err#22
1377/0x3ebc:      4571      10      7 sysctl([unknown, 3, 0, 0, 0, 0] (2), 0x7FF7BB40A250, 0x7FF7BB40A248, 0x7FF813739BA5, 0x15)     = 0 0
1377/0x3ebc:      4574       3      1 sysctl([CTL_KERN, 137, 0, 0, 0, 0] (2), 0x7FF7BB40A2E0, 0x7FF7BB40A2D8, 0x0, 0x0)      = 0 0
1377/0x3ebc:      4761     349    138 open("/tmp/ABC\0", 0x602, 0xDAA8)      = 3 0
1377/0x3ebc:      4781      20     18 write(0x3, "testtestov\0", 0xA)      = 10 0
1377/0x3ebc:      4845      64     63 close(0x3)     = 0 0
```

Фигура: Dtruss (откъс)



Фигура: Procmon (стек)

Обобщение

- Изложихме основните задачи, които решава една ОС.
- Уточнихме какво отличава ОС от библиотека за общо предназначение.
- Посочихме двата метода, чрез които ОС споделя ресурсите на машината.
- Разгледахме основната техника, използвана от ОС, за да изолират изпълнението на приложенията от хардуера.

За четене вкъщи

- Remzi H. Arpaci-Dusseau и Andrea C. Arpaci-Dusseau. *Introduction to Operating Systems*. https://pages.cs.wisc.edu/~remzi/OSTEP/intro.pdf. Chapter from Operating Systems: Three Easy Pieces, Version 1.10. 2018
- Remzi H. Arpaci-Dusseau и Andrea C. Arpaci-Dusseau. *Mechanism: Limited Direct Execution*. https://pages.cs.wisc.edu/~remzi/OSTEP/cpu-mechanisms.pdf. Chapter from Operating Systems: Three Easy Pieces, Version 1.10. 2018
- Computerphile. *Why Use Binary?* https://youtu.be/thrx3SBEpL8. Video, YouTube
- Computerphile. *Floating Point Numbers*. https://youtu.be/PZRI1IfStY0. Video, YouTube
- David Drysdale. *Anatomy of a system call, part 1*. https://lwn.net/Articles/604287/. Article from LWN.net. 2014

Край